15.7 Boyer-Moore Algorithm

Like the KMP algorithm, this also does some pre-processing and we call it *last function*. The algorithm scans the characters of the pattern from right to left beginning with the rightmost character. During the testing of a possible placement of pattern *P* in *T*, a mismatch is handled as follows: Let us assume that the current character being matched is $T[i] = c$ and the corresponding pattern character is $P[j]$. If *c* is not contained anywhere in *P*, then shift the pattern *P* completely past $T[i]$. Otherwise, shift *P* until an occurrence of character *c* in *P* gets aligned with $T[i]$. This technique avoids needless comparisons by shifting the pattern relative to the text.

The *last* function takes $O(m + |\sum|)$ time and the actual search takes $O(nm)$ time. Therefore the worst case running time of the Boyer-Moore algorithm is $O(nm + |\sum|)$. This indicates that the worst-case running time is quadratic, in the case of $n == m$, the same as the brute force algorithm.

- The Boyer-Moore algorithm is very fast on the large alphabet (relative to the length of the pattern).
- For the small alphabet, Boyer-Moore is not preferable.
- For binary strings, the KMP algorithm is recommended.
- For the very shortest patterns, the brute force algorithm is better.

15.8 Data Structures for Storing Strings

If we have a set of strings (for example, all the words in the dictionary) and a word which we want to search in that set, in order to perform the search operation faster, we need an efficient way of storing the strings. To store sets of strings we can use any of the following data structures.

- Hashing Tables
- Binary Search Trees
- Tries
- Ternary Search Trees

15.9 Hash Tables for Strings

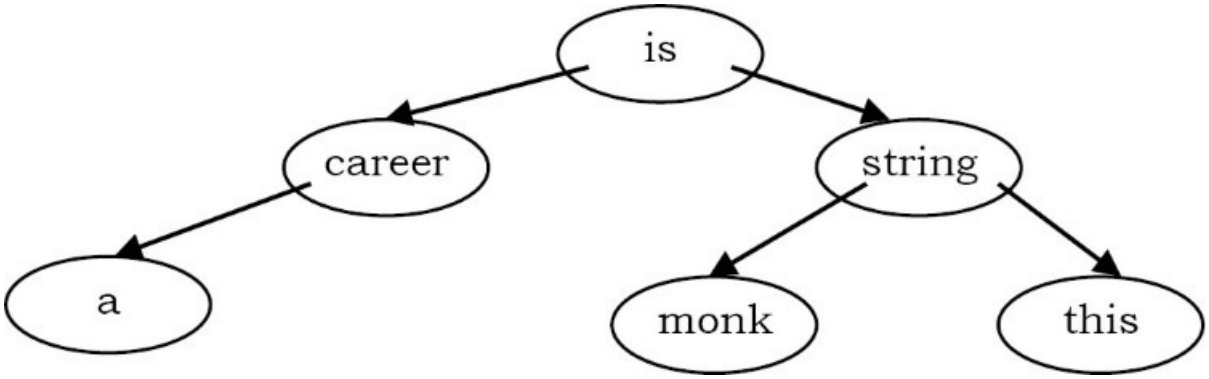
As seen in the *Hashing* chapter, we can use hash tables for storing the integers or strings. In this case, the keys are nothing but the strings. The problem with hash table implementation is that we lose the ordering information – after applying the hash function, we do not know where it will map to. As a result, some queries take more time. For example, to find all the words starting with the letter *"K"*, with hash table representation we need to scan the complete hash table. This is because the hash function takes the complete key, performs hash on it, and we do not know the location of each word.

15.10 Binary Search Trees for Strings

In this representation, every node is used for sorting the strings alphabetically. This is possible because the strings have a natural ordering: *A* comes before *B*, which comes before *C*, and so on. This is because words can be ordered and we can use a Binary Search Tree (BST) to store and retrieve them. For example, let us assume that we want to store the following strings using BSTs:

this is a career monk string

For the given string there are many ways of representing them in BST. One such possibility is shown in the tree below.



Issues with Binary Search Tree Representation

This method is good in terms of storage efficiency. But the disadvantage of this representation is that, at every node, the search operation performs the complete match of the given key with the node data, and as a result the time complexity of the search operation increases. So, from this we can say that BST representation of strings is good in terms of storage but not in terms of time.

15.11 Tries

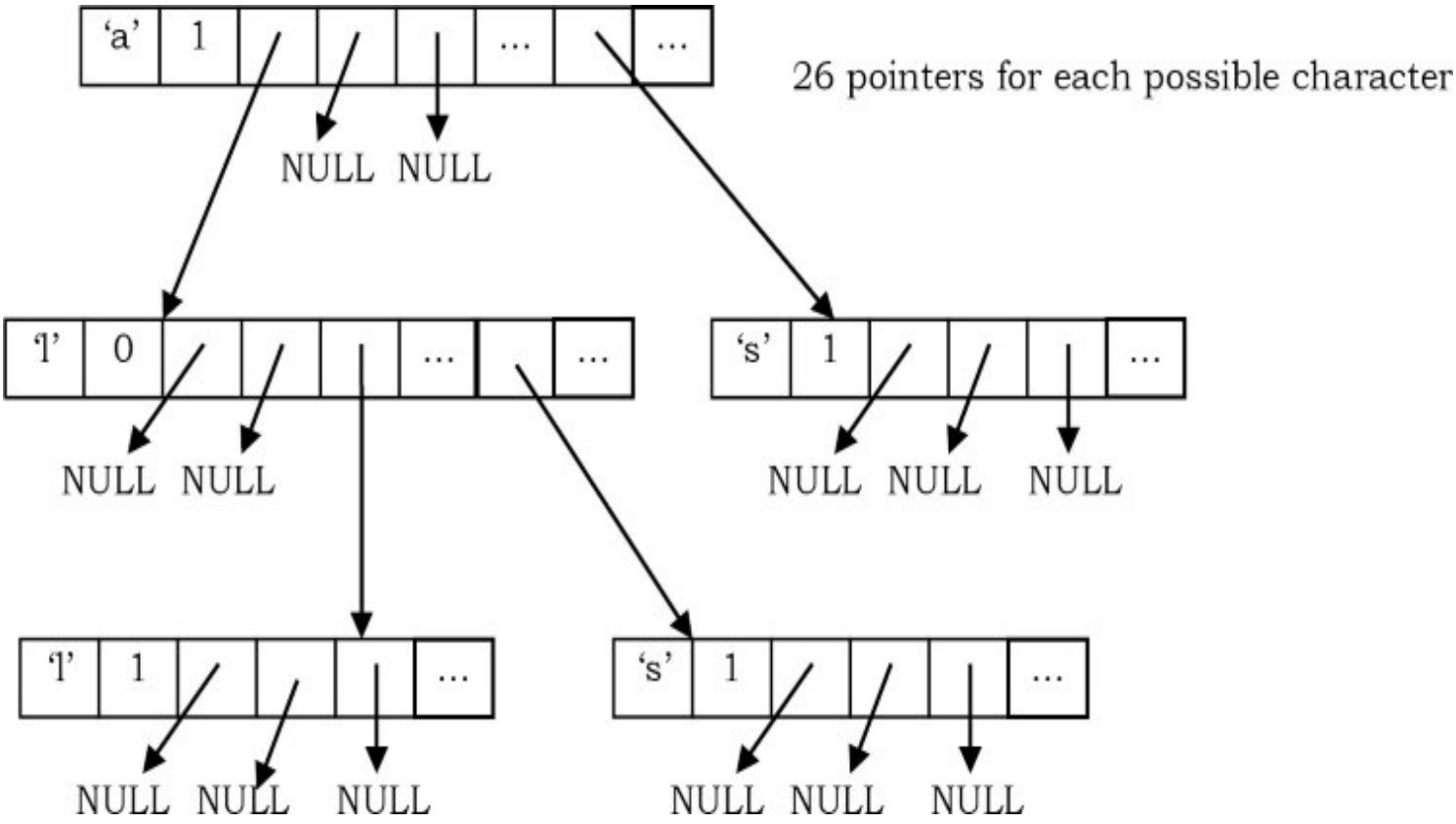
Now, let us see the alternative representation that reduces the time complexity of the search operation. The name *trie* is taken from the word re”trie”.

What is a Trie?

A *trie* is a tree and each node in it contains the number of pointers equal to the number of characters of the alphabet. For example, if we assume that all the strings are formed with English alphabet characters *“a”* to *“z”* then each node of the trie contains 26 pointers. A trie data structure can be declared as:

```
struct TrieNode {
    char data;                  // Contains the current node character.
    int is_End_Of_String;       // Indicates whether the string formed from root to
                                // current node is a string or not
    struct TrieNode *child[26]; // Pointers to other tri nodes
};
```

Suppose we want to store the strings *"a","all","als"*, and *"as""": trie* for these strings will look like:



Why Tries?

The tries can insert and find strings in O(*L*) time (where *L* represents the length of a single word). This is much faster than hash table and binary search tree representations.

Trie Declaration

The structure of the TrieNode has data (char), is_End_Of_String (boolean), and has a collection of child nodes (Collection of TrieNodes). It also has one more method called subNode(char). This method takes a character as argument and will return the child node of that character type if that is present. The basic element - TrieNode of a TRIE data structure looks like this:

```
struct TrieNode {
    char data;
    int is_End_Of_String;
    struct TrieNode *child[];
};
struct TrieNode *TrieNode subNode(struct TrieNode *root, char c){
    if(root! = NULL){
        for(int i=0; i < 26; i++){
            if(root.child[i]→data == c)
                return root.child[i];
        }
    }
    return NULL;
}
```

Now that we have defined our TrieNode, let's go ahead and look at the other operations of TRIE. Fortunately, the TRIE data structure is simple to implement since it has two major methods: insert() and search(). Let's look at the elementary implementation of both these methods.

Inserting a String in Trie

To insert a string, we just need to start at the root node and follow the corresponding path (path from root indicates the prefix of the given string). Once we reach the NULL pointer, we just need to create a skew of tail nodes for the remaining characters of the given string.

```
void InsertInTrie(struct TrieNode *root, char *word) {
    if(!*word) return;
    if(!root) {
        struct TrieNode *newNode = (struct TrieNode *) malloc (sizeof(struct TrieNode *));
          newNode→data=*word;
          for(int i =0; i<26; i++)
              newNode→child[i]=NULL;
          if(!*(word+1))
              newNode→is_End_Of_String = 1;
          else newNode→child[*word] = InsertInTrie(newNode→child[*word], word+1);
          return newNode;
    }
    root→child[*word] = InsertInTrie(root→child[*word], word+1);
    return  root;
}
```

Time Complexity: O(*L*), where *L* is the length of the string to be inserted.

Note: For real dictionary implementation, we may need a few more checks such as checking whether the given string is already there in the dictionary or not.

Searching a String in Trie

The same is the case with the search operation: we just need to start at the root and follow the pointers. The time complexity of the search operation is equal to the length of the given string that want to search.

```
int SearchInTrie(struct TrieNode *root, char *word) {
    if(!root)
        return -1;
    if(!*word) {
        if(root→ is_End_Of_String)
            return 1;
        else return -1;
    }
    if(root→data == *word)
        return  SearchInTrie(root→child[*word], word+1);
    else  return -1;
}
```

Time Complexity: O(*L*), where *L* is the length of the string to be searched.

Issues with Tries Representation

The main disadvantage of tries is that they need lot of memory for storing the strings. As we have seen above, for each node we have too many node pointers. In many cases, the occupancy of each node is less. The final conclusion regarding tries data structure is that they are faster but require huge memory for storing the strings.

Note: There are some improved tries representations called *trie compression techniques*. But, even with those techniques we can reduce the memory only at the leaves and not at the internal nodes.

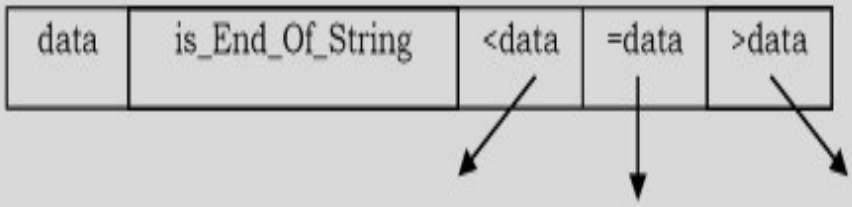
15.12 Ternary Search Trees

This representation was initially provided by Jon Bentley and Sedgewick. A ternary search tree takes the advantages of binary search trees and tries. That means it combines the memory

efficiency of BSTs and the time efficiency of tries.

Ternary Search Trees Declaration

```
struct TSTNode {
    char data;
    int is_End_Of_String;
    struct TSTNode *left;
    struct TSTNode *eq;
    struct TSTNode *right;
};
```

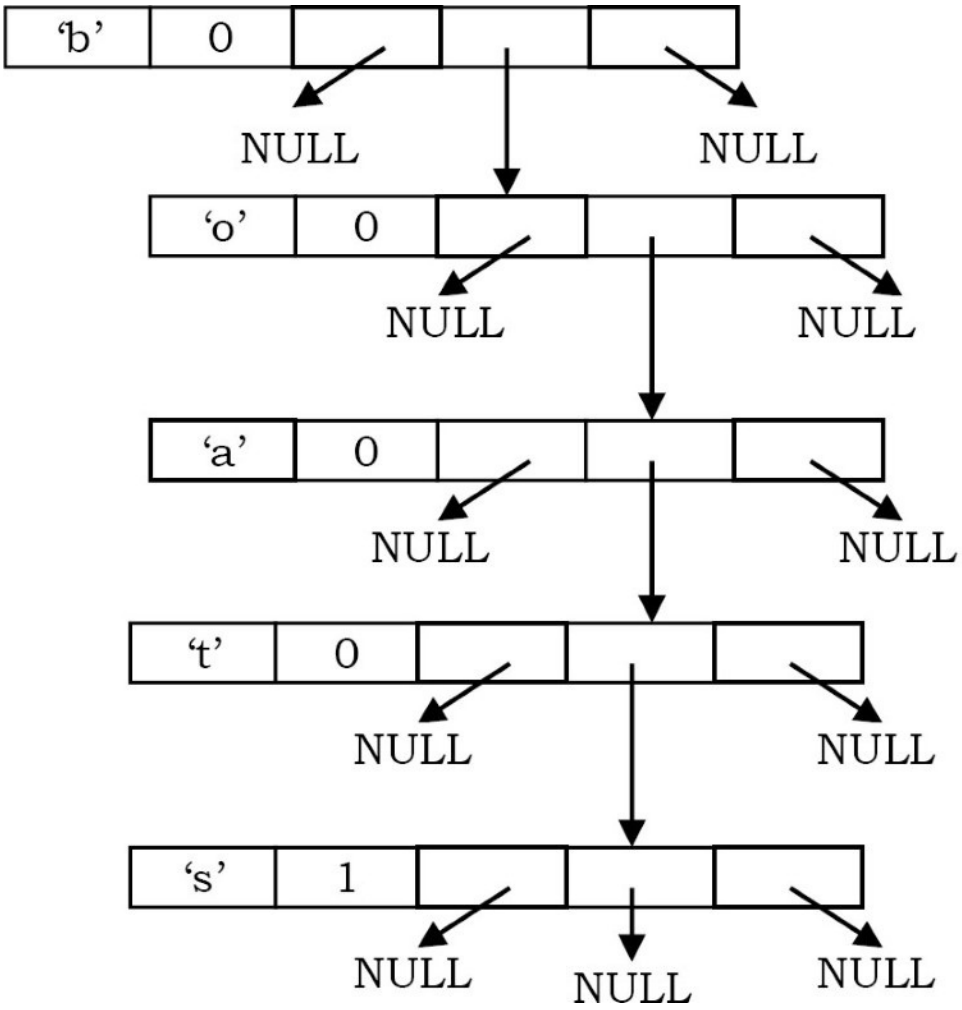


The Ternary Search Tree (TST) uses three pointers:

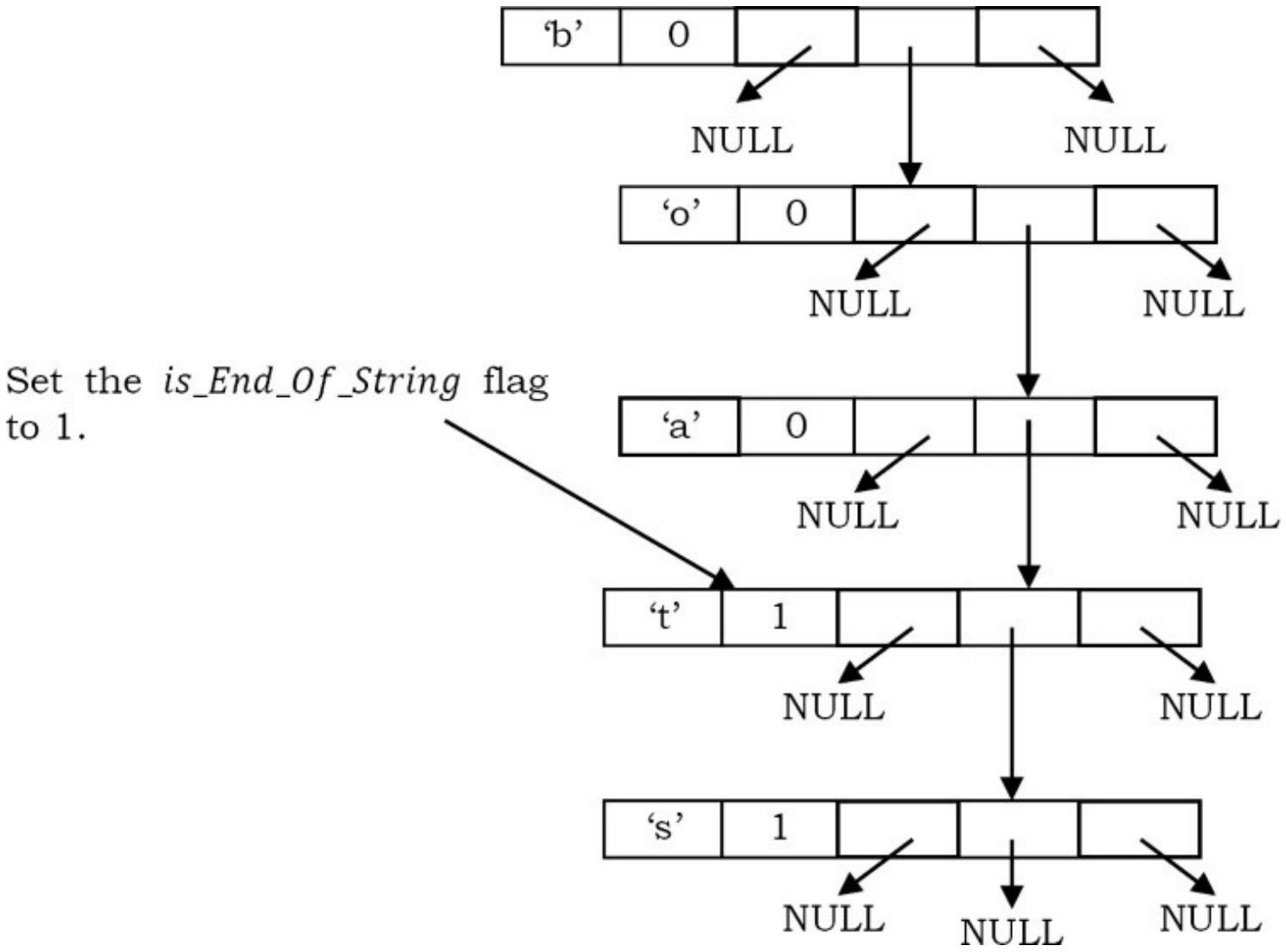
- The *left* pointer points to the TST containing all the strings which are alphabetically less than *data*.
- The *right* pointer points to the TST containing all the strings which are alphabetically greater than *data*.
- The *eq* pointer points to the TST containing all the strings which are alphabetically equal to *data*. That means, if we want to search for a string, and if the current character of the input string and the *data* of current node in TST are the same, then we need to proceed to the next character in the input string and search it in the subtree which is pointed by *eq*.

Inserting strings in Ternary Search Tree

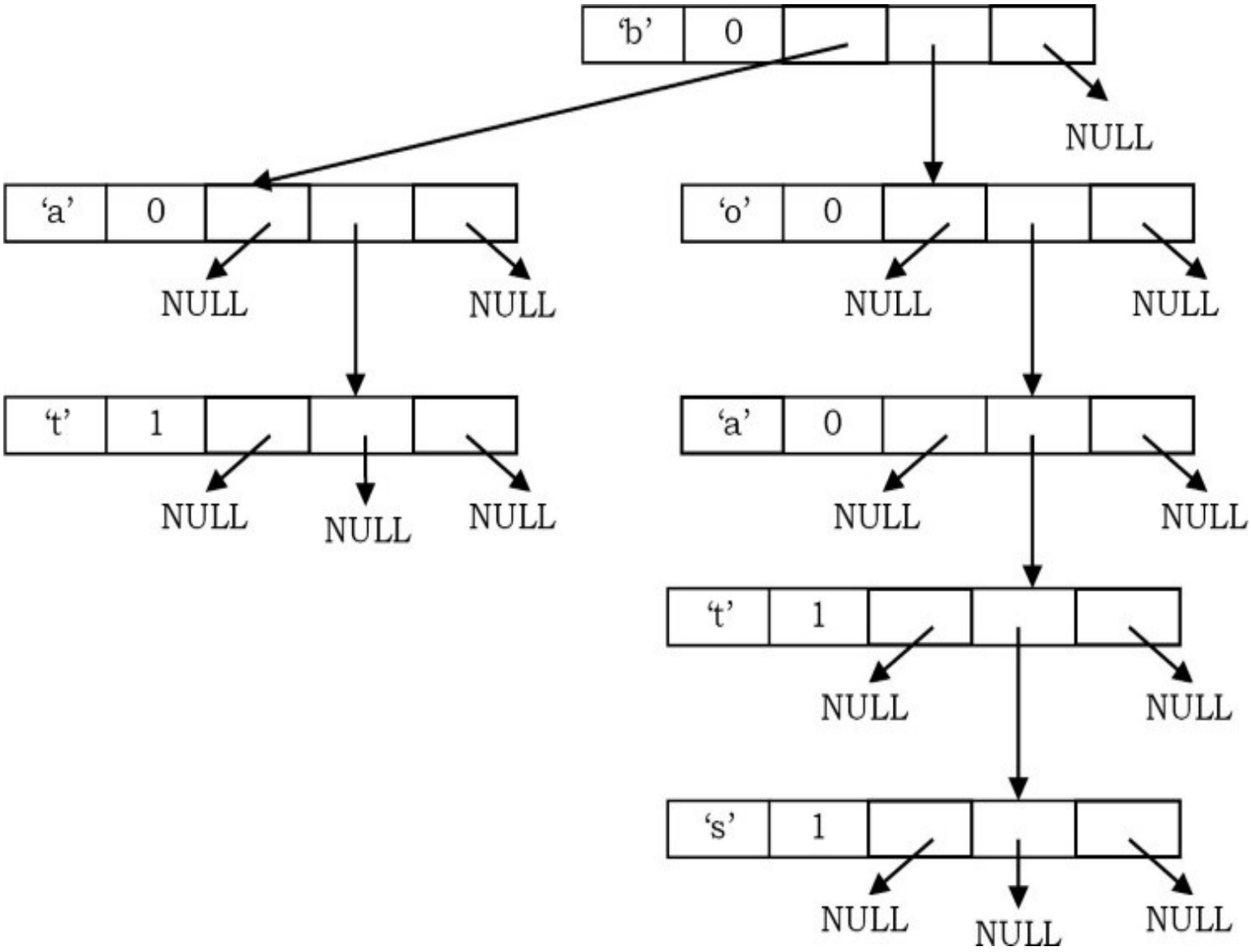
For simplicity let us assume that we want to store the following words in TST (also assume the same order): *boats*, *boat*, *bat* and *bats*. Initially, let us start with the *boats* string.



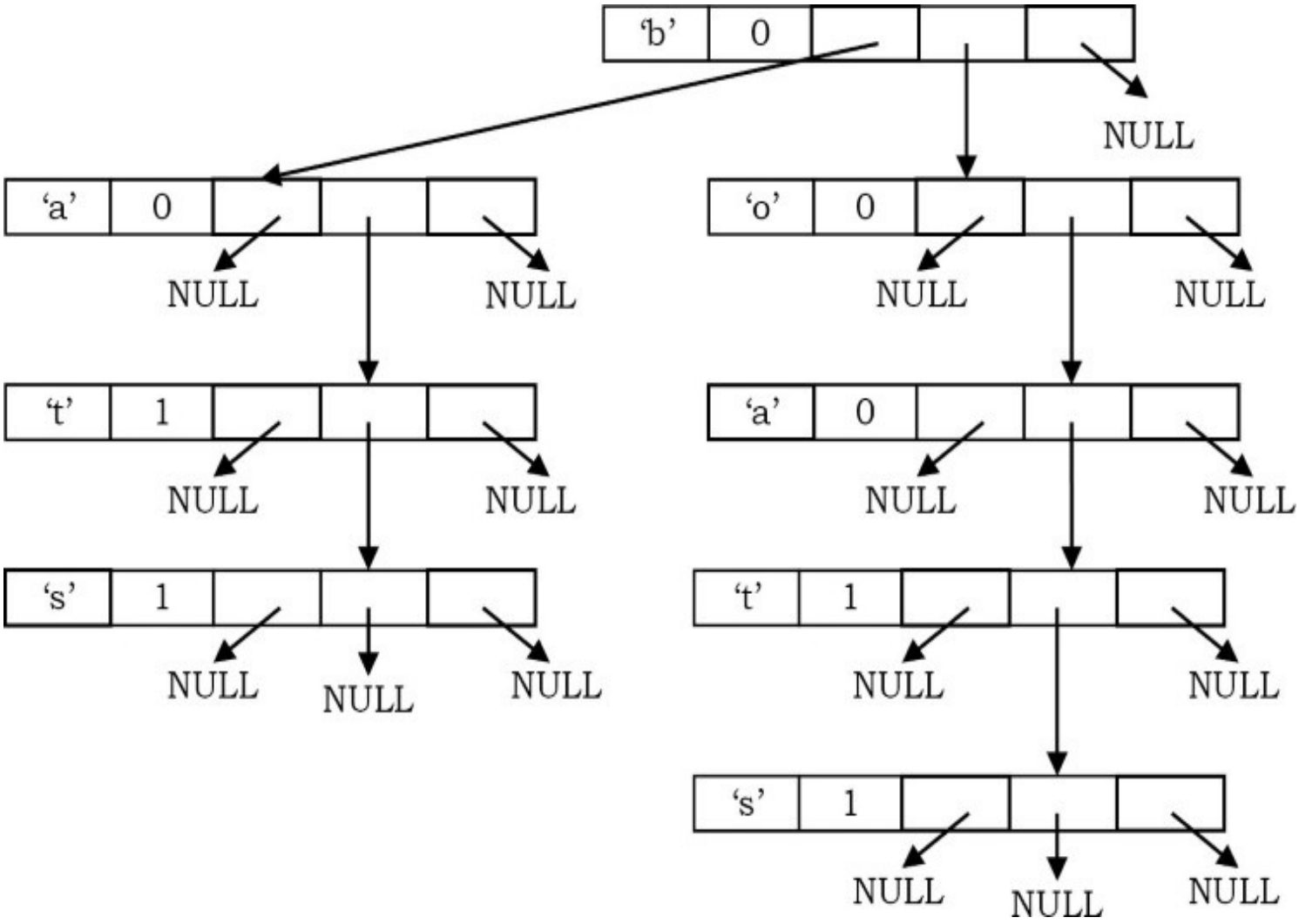
Now if we want to insert the string *boat*, then the TST becomes [the only change is setting the *is_End_Of_String* flag of *"t"* node to 1]:



Now, let us insert the next string: *bat*



Now, let us insert the final word: *bats*.



Based on these examples, we can write the insertion algorithm as below. We will combine the insertion operation of BST and tries.

```
struct TSTNode  *InsertInTST(struct TSTNode  *root, char *word) {
    if(root == NULL) {
            root = (struct TSTNode  *) malloc(sizeof(struct TSTNode));
            root→data = *word;
            root→is_End_Of_String = 1;
            root→left = root→eq = root→right = NULL;
    }
    if(*word < root→data)
            root→left = InsertInTST (root→left, word);
    else if(*word == root→data) {
            if(*(word+1))
                root→eq = InsertInTST (root→eq, word+1);
            else root→is_End_Of_String = 1;
    }
    else   root→right = InsertInTST (root→right, word);
    return root;
}
```

Time Complexity: O(*L*), where *L* is the length of the string to be inserted.

Searching in Ternary Search Tree

If after inserting the words we want to search for them, then we have to follow the same rules as that of binary search. The only difference is, in case of match we should check for the remaining characters (in *eq* subtree) instead of return. Also, like BSTs we will see both recursive and non-recursive versions of the search method.

```
int SearchInTSTRecursive(struct TSTNode  *root, char *word) {
    if(!root)
        return -1;
    if(*word < root→data)
        return SearchInTSTRecursive(root→left, word);
    else if(*word > root→data)
        return SearchInTSTRecursive(root→right, word);
    else {
         if(root→is_End_Of_String && *(word+1)==0)
            return 1;
         return SearchInTSTRecursive(root→eq, ++word);
    }
}
int SearchInTSTNon-Recursive(struct TSTNode  *root, char *word) {
    while (root) {
            if(*word < root→data)
                root = root→left;
            else if(*word == root→data) {
                if(root→is_End_Of_String && *(word+1) == 0)
                    return 1;
                word++;
                root = root→eq;
            }
            else root = root→right;
    }
    return -1;
}
```

Time Complexity: O(*L*), where *L* is the length of the string to be searched.

Displaying All Words of Ternary Search Tree

If we want to print all the strings of TST we can use the following algorithm. If we want to print them in sorted order, we need to follow the inorder traversal of TST.

```
char word[1024];
void DisplayAllWords(struct TSTNode  *root) {
    if(!root)
        return;
    DisplayAllWords(root→left);
    word[i] = root→data;
    if(root→is_End_Of_String) {
            word[i] = '\0';
            printf("%c", word);
    }
    i++;
    DisplayAllWords(root→eq);

    i--;
    DisplayAllWords(root→right);
}
```

Finding the Length of the Largest Word in TST

This is similar to finding the height of the BST and can be found as:

```
int MaxLengthOfLargestWordInTST(struct TSTNode  *root) {
    if(!root)
        return 0;
    return Max(MaxLengthOfLargestWordInTST(root→left),
            MaxLengthOfLargestWordInTST(root→eq)+1,
            MaxLengthOfLargestWordInTST(root→right)));
}
```

15.13 Comparing BSTs, Tries and TSTs

- Hash table and BST implementation stores complete the string at each node. As a result they take more time for searching. But they are memory efficient.
- TSTs can grow and shrink dynamically but hash tables resize only based on load factor.
- TSTs allow partial search whereas BSTs and hash tables do not support it.
- TSTs can display the words in sorted order, but in hash tables we cannot get the sorted order.
- Tries perform search operations very fast but they take huge memory for storing the string.

- TSTs combine the advantages of BSTs and Tries. That means they combine the memory efficiency of BSTs and the time efficiency of tries

15.14 Suffix Trees

Suffix trees are an important data structure for strings. With suffix trees we can answer the queries very fast. But this requires some preprocessing and construction of a suffix tree. Even though the construction of a suffix tree is complicated, it solves many other string-related problems in linear time.

Note: Suffix trees use a tree (suffix tree) for one string, whereas Hash tables, BSTs, Tries and TSTs store a set of strings. That means, a suffix tree answers the queries related to one string.

Let us see the terminology we use for this representation.

Prefix and Suffix

Given a string $T = T_1T_2 \ldots T_n$, the *prefix* of T is a string $T_1 \ldots T_i$ where i can take values from 1 to n. For example, if $T = banana$, then the prefixes of T are: *b, ba, ban, bana, banan, banana.*

Similarly, given a string $T = T_1T_2 \ldots T_n$, the *suffix* of T is a string $T_i \ldots T_n$ where i can take values from n to 1. For example, if $T = banana$, then the suffixes of T are: *a, na, ana, nana, anana, banana.*

Observation

From the above example, we can easily see that for a given text T and pattern P, the exact string matching problem can also be defined as:

- Find a suffix of T such that P is a prefix of this suffix *or*
- Find a prefix of T such that P is a suffix of this prefix.

Example: Let the text to be searched be $T = acebkkbac$ and the pattern be $P = kkb$. For this example, P is a prefix of the suffix *kkbac* and also a suffix of the prefix *acebkkb.*

What is a Suffix Tree?

In simple terms, the suffix tree for text T is a Trie-like data structure that represents the suffixes of T. The definition of suffix trees can be given as: A suffix tree for a n character string $T[1 \ldots n]$ is a rooted tree with the following properties.

- A suffix tree will contain n leaves which are numbered from 1 to n

- Each internal node (except root) should have at least 2 children
- Each edge in a tree is labeled by a nonempty substring of *T*
- No two edges of a node (children edges) begin with the same character
- The paths from the root to the leaves represent all the suffixes of *T*

The Construction of Suffix Trees

Algorithm

1. Let *S* be the set of all suffixes of *T*. Append $ to each of the suffixes.
2. Sort the suffixes in S based on their first character.
3. For each group S_c ($c \in \sum$):
   (i) If S_c group has only one element, then create a leaf node.
   (ii) Otherwise, find the longest common prefix of the suffixes in S_c group, create an internal node, and recursively continue with Step 2, S being the set of remaining suffixes from S_c after splitting off the longest common prefix.

For better understanding, let us go through an example. Let the given text be *T* = *tatat*. For this string, give a number to each of the suffixes.

Index	Suffix
1	$
2	*t$*
3	*at$*
4	*tat$*
5	*atat$*
6	*tatat$*

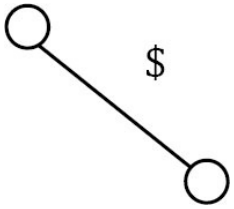
Now, sort the suffixes based on their initial characters.

Index	Suffix
1	$
3	*at$*
5	*atat$*
2	*t$*
4	*tat$*
6	*tatat$*

Group S_1 based on a
Group S_2 based on a
Group S_3 based on t

In the three groups, the first group has only one element. So, as per the algorithm, create a leaf

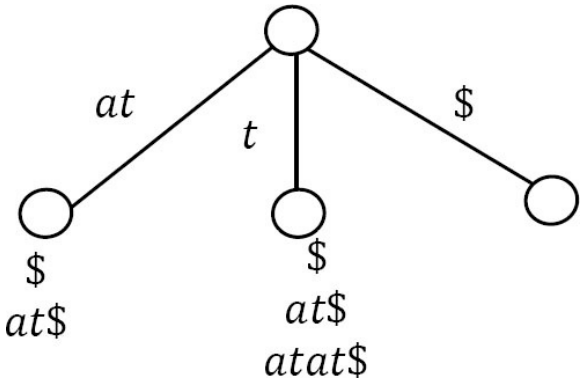
node for it, as shown below.



Now, for S_2 and S_3 (as they have more than one element), let us find the longest prefix in the group, and the result is shown below.

Group	Indexes for this group	Longest Prefix of Group Suffixes
S_2	3, 5	at
S_3	2, 4, 6	t

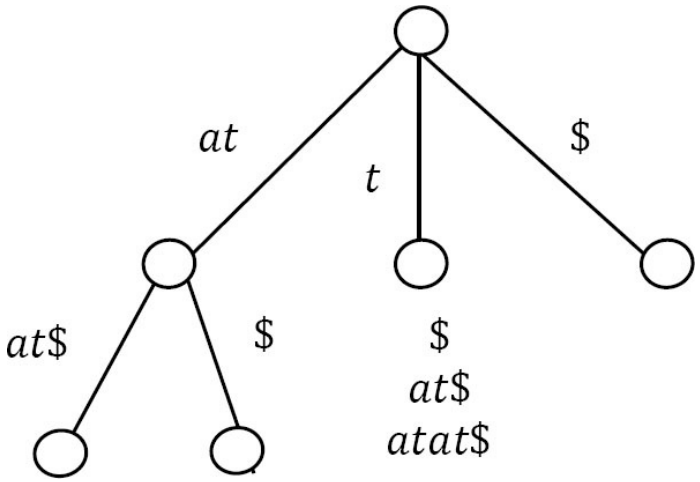
For S_2 and S_3, create internal nodes, and the edge contains the longest common prefix of those groups.



Now we have to remove the longest common prefix from the S_2 and S_3 group elements.

Group	Indexes for this group	Longest Prefix of Group Suffixes	Resultant Suffixes
S_2	3, 5	at	$\$, at\$$
S_3	2, 4, 6	t	$\$, at\$, atat\$$

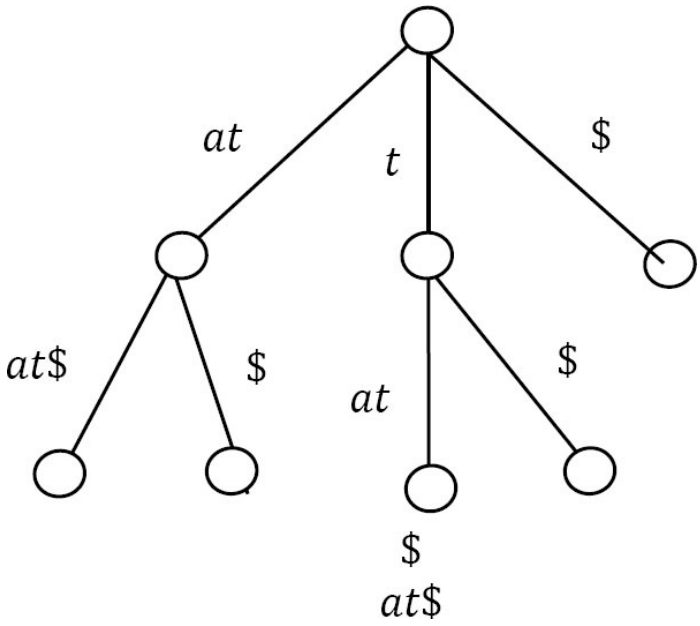
Out next step is solving S_2 and S_3 recursively. First let us take S_2. In this group, if we sort them based on their first character, it is easy to see that the first group contains only one element $, and the second group also contains only one element, *at*$. Since both groups have only one element, we can directly create leaf nodes for them.



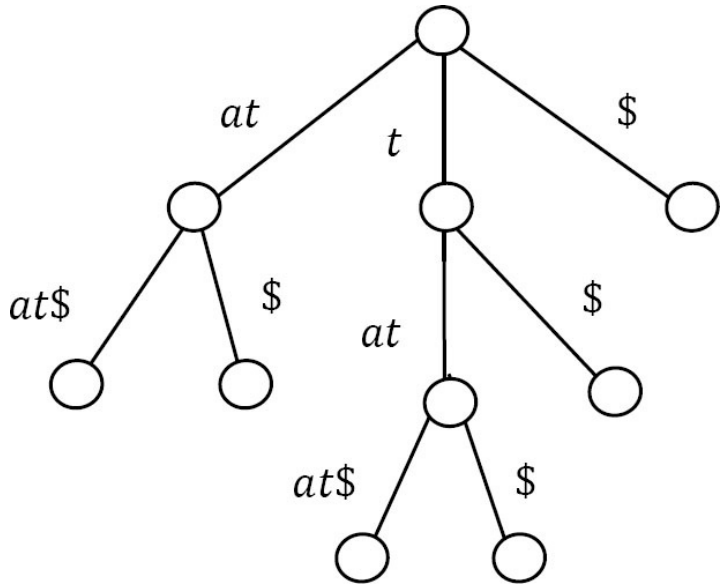
At this step, both S_1 and S_2 elements are done and the only remaining group is S_3. As similar to earlier steps, in the S_3 group, if we sort them based on their first character, it is easy to see that there is only one element in the first group and it is \$. For S_3 remaining elements, remove the longest common prefix.

Group	Indexes for this group	Longest Prefix of Group Suffixes	Resultant Suffixes
S_3	4, 6	at	\$, at\$

In the S_3 second group, there are two elements: \$ and *at*\$. We can directly add the leaf nodes for the first group element \$. Let us add S_3 subtree as shown below.



Now, S_3 contains two elements. If we sort them based on their first character, it is easy to see that there are only two elements and among them one is \$ and other is *at*\$. We can directly add the leaf nodes for them. Let us add S_3 subtree as shown below.



Since there are no more elements, this is the completion of the construction of the suffix tree for string $T = tatat$. The time-complexity of the construction of a suffix tree using the above algorithm is $O(n^2)$ where n is the length of the input string because there are n distinct suffixes. The longest has length n, the second longest has length $n - 1$, and so on.

Note:

- There are $O(n)$ algorithms for constructing suffix trees.
- To improve the complexity, we can use indices instead of string for branches.

Applications of Suffix Trees

All the problems below (but not limited to these) on strings can be solved with suffix trees very efficiently (for algorithms refer to *Problems* section).

- **Exact String Matching:** Given a text T and a pattern P, how do we check whether P appears in T or not?
- **Longest Repeated Substring:** Given a text T how do we find the substring of T that is the maximum repeated substring?
- **Longest Palindrome:** Given a text T how do we find the substring of T that is the longest palindrome of T?
- **Longest Common Substring:** Given two strings, how do we find the longest common substring?
- **Longest Common Prefix:** Given two strings $X[i \ldots n]$ and $Y[j \ldots m]$,how do we find the longest common prefix?
- How do we search for a regular expression in given text T?
- Given a text T and a pattern P, how do we find the first occurrence of P in T?

15.15 String Algorithms: Problems & Solutions

Problem-1 Given a paragraph of words, give an algorithm for finding the word which appears the maximum number of times. If the paragraph is scrolled down (some words disappear from the first frame, some words still appear, and some are new words), give the maximum occurring word. Thus, it should be dynamic.

Solution: For this problem we can use a combination of priority queues and tries. We start by creating a trie in which we insert a word as it appears, and at every leaf of trie. Its node contains that word along with a pointer that points to the node in the heap [priority queue] which we also create. This heap contains nodes whose structure contains a *counter*. This is its frequency and also a pointer to that leaf of trie, which contains that word so that there is no need to store the word twice.

Whenever a new word comes up, we find it in trie. If it is already there, we increase the frequency of that node in the heap corresponding to that word, and we call it heapify. This is done so that at any point of time we can get the word of maximum frequency. While scrolling, when a word goes out of scope, we decrement the counter in heap. If the new frequency is still greater than zero, heapify the heap to incorporate the modification. If the new frequency is zero, delete the node from heap and delete it from trie.

Problem-2 Given two strings, how can we find the longest common substring?

Solution: Let us assume that the given two strings are T_1 and T_2. The longest common substring of two strings, T_1 and T_2, can be found by building a generalized suffix tree for T_1 and T_2. That means we need to build a single suffix tree for both the strings. Each node is marked to indicate if it represents a suffix of T_1 or T_2 or both. This indicates that we need to use different marker symbols for both the strings (for example, we can use \$ for the first string and # for the second symbol). After constructing the common suffix tree, the deepest node marked for both T_1 and T_2 represents the longest common substring.

Another way of doing this is: We can build a suffix tree for the string $T_1\$T_2\#$. This is equivalent to building a common suffix tree for both the strings.

Time Complexity: O($m + n$), where m and n are the lengths of input strings T_1 and T_2.

Problem-3 **Longest Palindrome:** Given a text T how do we find the substring of T which is the longest palindrome of T?

Solution: The longest palindrome of $T[1..n]$ can be found in O(n) time. The algorithm is: first build a suffix tree for $T\$reverse(T)\#$ or build a generalized suffix tree for T and $reverse(T)$. After building the suffix tree, find the deepest node marked with both \$ and #. Basically it means find the longest common substring.

Problem-4 Given a string (word), give an algorithm for finding the next word in the dictionary.

Solution: Let us assume that we are using Trie for storing the dictionary words. To find the next

word in Tries we can follow a simple approach as shown below. Starting from the rightmost character, increment the characters one by one. Once we reach *Z*, move to the next character on the left side.

Whenever we increment, check if the word with the incremented character exists in the dictionary or not. If it exists, then return the word, otherwise increment again. If we use *TST*, then we can find the inorder successor for the current word.

Problem-5 Give an algorithm for reversing a string.

Solution:

```
//If the str is editable
char *ReversingString(char str[]) {
        char temp, start, end;
        if(str == NULL || *str == '\0')
                return str;
        for (end = 0; str[end]; end++);
        end--;
        for (start = 0; start < end; start++, end--) {
                temp = str[start]; str[start] = str[end]; str[end] = temp;
        }
        return str;
}
```

Time Complexity: O(*n*), where *n* is the length of the given string. Space Complexity: O(*n*).

Problem-6 If the string is not editable, how do we create a string that is the reverse of the given string?

Solution: If the string is not editable, then we need to create an array and return the pointer of that.

```
//If str is a const string (not editable)
char* ReversingString(char* str) {
        int start, end, len;
        char temp, *ptr=NULL;
        len=strlen(str);
        ptr=malloc(sizeof(char)*(len+1));
        ptr=strcpy(ptr,str);
        for (start=0, end=len-1; start<=end; start++, end--) {        //Swapping
                temp=ptr[start]; ptr[start]=ptr[end]; ptr[end]=temp;
        }
        return ptr;
}
```

Time Complexity: $O\left(\frac{n}{2}\right) \approx O(n)$, where *n* is the length of the given string. Space Complexity: O(1).

Problem-7 Can we reverse the string without using any temporary variable?

Solution: Yes, we can use XOR logic for swapping the variables.

```
char* ReversingString(char *str) {
        int start = 0, end= strlen(str)-1;
        while( start<end ) {
                str[start] ^= str[end];     str[end]  ^=  str[start];   str[start] ^= str[end];
                ++start;
                --end;
        }
        return str;
}
```

Time Complexity: $O\left(\frac{n}{2}\right) \approx O(n)$, where *n* is the length of the given string. Space Complexity: O(1).

Problem-8 Given a text and a pattern, give an algorithm for matching the pattern in the text. Assume ? (single character matcher) and * (multi character matcher) are the wild card characters.

Solution: Brute Force Method. For efficient method, refer to the theory section.

```
int PatternMatching(char *text, char *pattern) {
    if(*pattern == 0)
        return 1;
    if(*text == 0)
        return *p == 0;
    if('?' == *pattern)
        return PatternMatching(text+1,pattern+1) || PatternMatching(text,pattern+1);
    if('*' == *pattern)
        return PatternMatching(text+1,pattern) || PatternMatching(text,pattern+1);
    if(*text == *pattern)
        return PatternMatching(text+1,pattern+1);
    return -1;
}
```

Time Complexity: O(*mn*), where *m* is the length of the text and *n* is the length of the pattern.

Space Complexity: O(1).

Problem-9 Give an algorithm for reversing words in a sentence.

Example: Input: “This is a Career Monk String”, Output: “String Monk Career a is This”

Solution: Start from the beginning and keep on reversing the words. The below implementation assumes that ‘ ‘ (space) is the delimiter for words in given sentence.

```
void ReverseWordsInSentences(char *text) {
    int wordStart, wordEnd, length;
    length = strlen(text);
    ReversingString(text, 0, length-1);
    for(wordStart = wordEnd = 0; wordEnd < length; wordEnd ++) {
        if(text[wordEnd] ! = ' ') {
            wordStart = wordEnd;
            while (text[wordEnd] != ' ' && wordEnd < length)
                wordEnd ++;
            wordEnd--;
            ReversingString(text, wordStart, wordEnd); //Found current word, reverse it now.
        }
    }
}
void ReversingString(char text[], int start, int end) {
    for (char temp; start < end; start++, end--) {
        temp = str[end];
        str[end] = str[start];
        strstart] = temp;
    }
}
```

Time Complexity: O(2n) ≈ O(n), where n is the length of the string. Space Complexity: O(1).

Problem-10 **Permutations of a string [anagrams]:** Give an algorithm for printing all possible permutations of the characters in a string. Unlike combinations, two permutations are considered distinct if they contain the same characters but in a different order. For simplicity assume that each occurrence of a repeated character is a distinct character. That is, if the input is "aaa", the output should be six repetitions of "aaa". The permutations may be output in any order.

Solution: The solution is reached by generating n! strings, each of length n, where n is the length of the input string.

```
void Permutations(int depth, char *permutation, int *used, char *original) {
    int length = strlen(original);
    if(depth == length)
        printf("%s",permutation);
    else {
        for (int i = 0; i < length; i++) {
            if(!used[i]) {
                used[i] = 1;
                permutation[depth] = original[i];
                Permutations(depth + 1, permutation, used, original);
                used[i] = 0;
            }
        }
    }
}
```

Problem-11 Combinations Combinations of a String: Unlike permutations, two combinations are considered to be the same if they contain the same characters, but may be in a different order. Give an algorithm that prints all possible combinations of the characters in a string. For example, *"ac"* and *"ab"* are different combinations from the input string *"abc"*, but *"ab"* is the same as *"ba"*.

Solution: The solution is achieved by generating $n!/r!\ (n - r)!$ strings, each of length between 1 and n where n is the length of the given input string.

Algorithm:

For each of the input characters

a. Put the current character in output string and print it.
b. If there are any remaining characters, generate combinations with those remaining characters.

```
void Combinations(int depth, char *combination, int start, char *original) {
    int length = strlen(original);
    for (int i = start; i < length; i++) {
        combination[depth] = original[i];
        combination[depth +1] = '\0';
        printf("%s", combination);
        if(i < length -1)
            Combinations(depth + 1, combination, start + 1, original);
    }
}
```

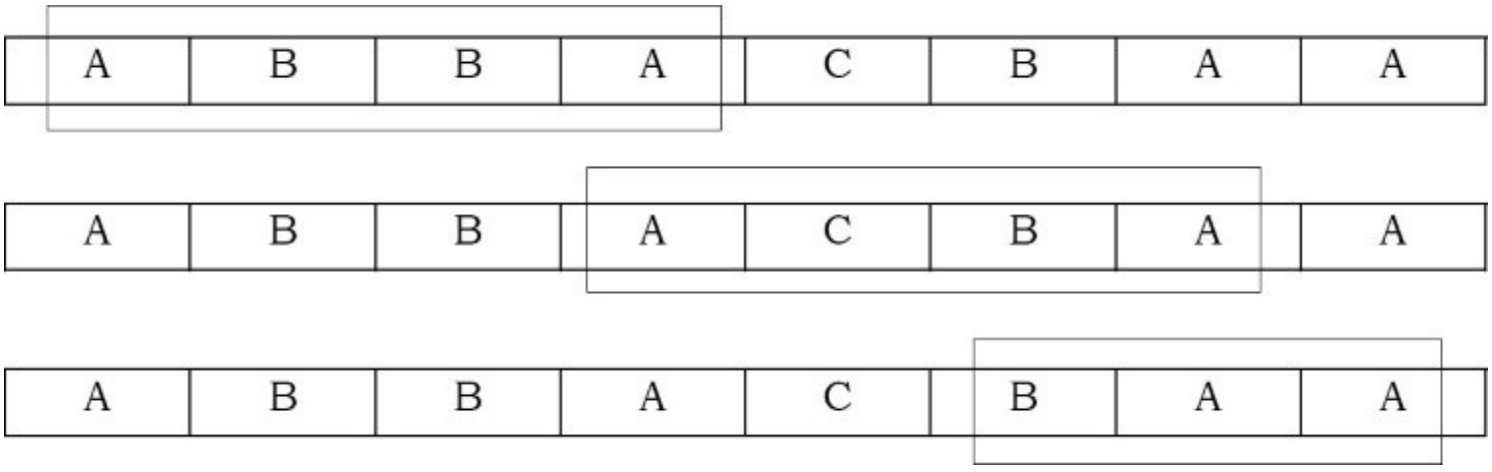
Problem-12 Given a string "ABCCBCBA", give an algorithm for recursively removing the adjacent characters if they are the same. For example, ABCCBCBA nnnnnn> ABBCBA->ACBA

Solution: First we need to check if we have a character pair; if yes, then cancel it. Now check for next character and previous element. Keep canceling the characters until we either reach the start of the array, reach the end of the array, or don't find a pair.

```
void RremoveAdjacentPairs(char* str) {
    int len = strlen(str), i, j = 0;
    for (i=1; i <= len; i++) {
        while ((str[i] == str[j]) && (j >= 0)){        //Cancel pairs
            i++;
            j--;
        }
        str[++j] = str[i];
    }
    return;
}
```

Problem-13 Given a set of characters *CHARS* and a input string *INPUT*, find the minimum window in *str* which will contain all the characters in *CHARS* in complexity O(*n*). For example, *INPUT = ABBACBAA* and *CHARS = AAB* has the minimum window *BAA*.

Solution: This algorithm is based on the sliding window approach. In this approach, we start from the beginning of the array and move to the right. As soon as we have a window which has all the required elements, try sliding the window as far right as possible with all the required elements. If the current window length is less than the minimum length found until now, update the minimum length. For example, if the input array is *ABBACBAA* and the minimum window should cover characters *AAB*, then the sliding window will move like this:



Algorithm: The input is the given array and chars is the array of characters that need to be found.

1 Make an integer array shouldfind[] of len 256. The i^{th} element of this array will have the count of how many times we need to find the element of ASCII value *i*.
2 Make another array hasfound of 256 elements, which will have the count of the required elements found until now.
3 Count <= 0
4 While input[i]
    a. If input[i] element is not to be found → continue
    b. If input[i] element is required => increase count by 1.
    c. If count is length of chars[] array, slide the window as much right as possible.
    d. If current window length is less than min length found until now, update min length.

```
#define MAX 256
void MinLengthWindow(char input[], char chars[]) {
        int shouldfind[MAX] = {0,}, hasfound[MAX] = {0,};
        int j=0, cnt = 0, start=0, finish, minwindow = INT_MAX;
        int charlen = strlen(chars), iplen = strlen(input);
        for (int i=0; i< charlen; i++)
                shouldfind[chars[i]] += 1;
        finish = iplen;
        for (int i=0; i< iplen; i++) {
                if(!shouldfind[input[i]])
                        continue;
                hasfound[input[i]] += 1;
                if(shouldfind[input[i]] >= hasfound[input[i]])
                        cnt++;
                if(cnt == charlen) {
                        while (shouldfind[input[j]] == 0 || hasfound[input[j]] > shouldfind[input[j]])          {
                                if(hasfound[input[j]] > shouldfind[input[j]])
                                        hasfound[input[j]]--;
                                j++;
                        }
                        if(minwindow > (i - j +1)) {
                                minwindow = i - j +1;
                                finish = i;
                                start = j;
                        }
                }
        }
        printf("Start:%d and Finish: %d", start, finish);
}
```

Complexity: If we walk through the code, *i* and *j* can traverse at most *n* steps (where *n* is the input

size) in the worst case, adding to a total of 2*n* times. Therefore, time complexity is O(*n*).

Problem-14 We are given a 2D array of characters and a character pattern. Give an algorithm to find if the pattern is present in the 2D array. The pattern can be in any order (all 8 neighbors to be considered) but we can't use the same character twice while matching. Return 1 if match is found, 0 if not. For example: Find "MICROSOFT" in the below matrix.

A	C	P	**R**	C
X	**S**	**O**	P	**C**
V	**O**	V	N	**I**
W	G	**F**	**M**	N
Q	A	**T**	I	T

Solution: Manually finding the solution of this problem is relatively intuitive; we just need to describe an algorithm for it. Ironically, describing the algorithm is not the easy part.

How do we do it manually? First we match the first element, and when it is matched we match the second element in the 8 neighbors of the first match. We do this process recursively, and when the last character of the input pattern matches, return true.

During the above process, take care not to use any cell in the 2D array twice. For this purpose, you mark every visited cell with some sign. If your pattern matching fails at some point, start matching from the beginning (of the pattern) in the remaining cells. When returning, you unmark the visited cells.

Let's convert the above intuitive method into an algorithm. Since we are doing similar checks for pattern matching every time, a recursive solution is what we need. In a recursive solution, we need to check if the substring passed is matched in the given matrix or not. The condition is not to use the already used cell, and to find the already used cell, we need to add another 2D array to the function (or we can use an unused bit in the input array itself.) Also, we need the current position of the input matrix from where we need to start. Since we need to pass a lot more information than is actually given, we should be having a wrapper function to initialize the extra information to be passed.

Algorithm:

```
If we are past the last character in the pattern
        Return true
If we get a used cell again
        Return false if we got past the 2D matrix
        Return false
If searching for first element and cell doesn't match
        FindMatch with next cell in row-first order (or column-first order)
```

Otherwise if character matches
    mark this cell as used
    res = FindMatch with next position of pattern in 8 neighbors
    mark this cell as unused
    Return res
Otherwise
    Return false

```
#define MAX 100
boolean FindMatch_wrapper(char mat[MAX][MAX], char *pat, int nrow, int ncol) {
      if(strlen(pat) > nrow*ncol) return false;
      int used[MAX][MAX] = {{0,},};
      return FindMatch(mat, pat, used, 0, 0, nrow, ncol, 0);
}
//level: index till which pattern is matched & x, y: current position in 2D array
boolean FindMatch(char mat[MAX][MAX], char *pat, int used[MAX][MAX],
                                        int x, int y, int nrow, int ncol, int level) {
   if(level == strlen(pat)) //pattern matched
      return true;
   if(nrow == x || ncol == y)   return false;
   if(used[x][y])        return false;
   if(mat[x][y] != pat[level] && level == 0) {
      if(x < (nrow - 1))
         return FindMatch(mat, pat, used, x+1, y, nrow, ncol, level); //next element in same row
      else if(y < (ncol - 1))
         return FindMatch(mat, pat, used, 0, y+1, nrow, ncol, level); //first element from same column
      else  return false;
   }
   else if(mat[x][y] == pat[level]) {
      boolean res;
      used[x][y] = 1;      //marking this cell as used
      //finding subpattern in 8 neighbors
      res = (x > 0  ? FindMatch(mat, pat, used, x-1, y, nrow, ncol, level+1) :   false) ||
         (res = x < (nrow - 1)  ? FindMatch(mat, pat, used, x+1, y, nrow, ncol, level+1) :   false) ||
         (res = y > 0 ? FindMatch(mat, pat, used, x, y-1, nrow, ncol, level+1) :   false) ||
         (res = y < (ncol - 1)  ? FindMatch(mat, pat, used, x, y+1, nrow, ncol, level+1) :   false) ||
          (res = x < (nrow - 1)&&(y < ncol -1)?FindMatch(mat, pat, used, x+1, y+1, nrow, ncol, level+1) :  false) ||
         (res = x < (nrow - 1) && y > 0  ?  FindMatch(mat, pat, used, x+1, y-1, nrow, ncol, level+1) : false) ||
         (res = x > 0 && y < (ncol - 1) ?    FindMatch(mat, pat, used, x-1, y+1, nrow, ncol, level+1) : false) ||
         (res = x > 0 && y > 0 ? FindMatch(mat, pat, used, x-1, y-1, nrow, ncol, level+1) : false);
      used[x][y] = 0;          //marking this cell as unused
      return res;
   }
   else  return false;
}
```

Problem-15 Given two strings str1 and *str*2, write a function that prints all interleavings of the given two strings. We may assume that all characters in both strings are different. Example: Input: *str*1 = "AB", *str*2 = "CD" and Output: ABCD ACBD ACDB CABD

CADB CDAB. An interleaved string of given two strings preserves the order of characters in individual strings. For example, in all the interleavings of above first example, ‘A’ comes before ‘B’ and ‘C comes before ‘D’.

Solution: Let the length of *str*1 be *m* and the length of *str*2 be *n*. Let us assume that all characters in *str*1 and *str*2 are different. Let Count(*m,n*) be the count of all interleaved strings in such strings. The value of Count(*m,n*) can be written as following.

```
Count(m, n) = Count(m-1, n) + Count(m, n-1)
Count(1, 0) = 1 and Count(1, 0) = 1
```

To print all interleavings, we can first fix the first character of strl[0..m-1] in output string, and recursively call for str1[1..m-1] and str2[0..n-1]. And then we can fix the first character of str2[0..n-1] and recursively call for str1[0..m-1] and str2[1..n-1].

```
void PrintInterleavings(char *str1, char *str2, char *iStr, int m, int n, int i){
   // Base case: If all characters of str1 & str2 have been included in output string,
   // then print the output string
   if ( m==0 && n ==0 )
      printf("%s\n", iStr) ;
   // If some characters of str1 are left to be included, then include the
   // first character from the remaining characters and recur for rest
   if ( m != 0 )   {
      iStr[i] = str1[0];
      PrintInterleavings(str1 + 1, str2, iStr, m-1, n, i+1);
   }
   // If some characters of str2 are left to be included, then include the
   // first character from the remaining characters and recur for rest
   if ( n != 0 ) {
      iStr[i] = str2[0];
      PrintInterleavings(str1, str2+1, iStr, m, n-1, i+1);
   }
}
// Allocates memory for output string and uses PrintInterleavings() for printing all interleaving's
void Print(char *str1, char *str2, int m, int n){
  // allocate memory for the output string
  char *iStr= (char*)malloc((m+n+1)*sizeof(char));
        // Set the terminator for the output string
  iStr[m+n] = '\0';
  // print all interleaving's using PrintInterleavings()
  PrintInterleavings(str1, str2, iStr, m, n, 0);
  free(iStr);
}
```

Problem-16 Given a matrix with size $n \times n$ containing random integers. Give an algorithm which checks whether rows match with a column(s) or not. For example, if i^{th} row matches with j^{th} column, and i^{th} row contains the elements - [2,6,5,8,9]. Then;''[1] column would also contain the elements - [2,6,5,8,9].

Solution: We can build a trie for the data in the columns (rows would also work). Then we can compare the rows with the trie. This would allow us to exit as soon as the beginning of a row does not match any column (backtracking). Also this would let us check a row against all columns in one pass.

If we do not want to waste memory for empty pointers then we can further improve the solution by constructing a suffix tree.

Problem-17 Write a method to replace all spaces in a string with '%20'. Assume string has sufficient space at end of string to hold additional characters.

Solution: Find the number of spaces. Then, starting from end (assuming string has enough space), replace the characters. Starting from end reduces the overwrites.

```
void encodeSpaceWithString(char* A){
    char *space = "%20";
    int stringLength = strlen(A);
    if(stringLength ==0){
        return;
    }
    int i, numberOfSpaces = 0;
    for(i = 0; i < stringLength; i++){
        if(A[i] == ' ' || A[i] == '\t'){
            numberOfSpaces ++;
        }
    }
    if(!numberOfSpaces)
        return;
    int newLength = len + numberOfSpaces * 2;
    A[newLength] = '\0';
    for(i = stringLength-1; i >= 0; i--){
        if(A[i] == ' ' || A[i] == '\t'){
            A[newLength--] = '0';
            A[newLength--] = '2';
            A[newLength--] = '%';
        }
        else{
            A[newLength--] = A[i];
        }
    }
}
```

Time Complexity: O(*n*). Space Complexity: O(1). Here, we do not have to worry about the space needed for extra characters.

Problem-18 **Running length encoding:** Write an algorithm to compress the given string by using the count of repeated characters and if new corn-pressed string length is not smaller than the original string then return the original string.

Solution:

With extra space of O(2):

```
string CompressString(string inputStr){
        char last = inputStr.at(0);
        int size = 0, count = 1;
        char temp[2];
        string str;
        for (int i = 1; i < inputStr.length(); i++){
                if(last == inputStr.at(i))
                        count ++;
                else{
                        itoa(count, temp, 10);
                        str += last;
                        str += temp;
                        last = inputStr.at(i);
                        count = 1;
                }
        }
        str = str + last + temp;
        // If the compresed string size is greater than input string, return input string
        if(str.length() >= inputStr.length())
                return inputStr;
        else return str;
}
```

Time Complexity: O(*n*). Space Complexity: O(1), but it uses a temporary array of size two.

Without extra space (*inplace*):

```
char CompressString(char *inputStr, char currentChar, int lengthIndex, int& countChar, int& index){
	if(lengthIndex == -1)
		return currentChar;
	char lastChar = CompressString(inputStr, inputStr[lengthIndex], lengthIndex-1, countChar, index);
	if(lastChar == currentChar)
		countChar++;
	else {
		inputStr[index++] = lastChar;
		for(int i =0; i< NumToString(countChar).length(); i++)
			inputStr[index++] = NumToString(countChar).at(i);
		countChar = 1;
	}
	return currentChar;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

CHAPTER

16

ALGORITHMS DESIGN TECHNIQUES

16.1 Introduction

In the previous chapters, we have seen many algorithms for solving different kinds of problems. Before solving a new problem, the general tendency is to look for the similarity of the current problem to other problems for which we have solutions. This helps us in getting the solution easily.

In this chapter, we will see different ways of classifying the algorithms and in subsequent chapters we will focus on a few of them (Greedy, Divide and Conquer, Dynamic Programming).

16.2 Classification

There are many ways of classifying algorithms and a few of them are shown below:

- Implementation Method
- Design Method
- Other Classifications

16.3 Classification by Implementation Method

Recursion or Iteration

A *recursive* algorithm is one that calls itself repeatedly until a base condition is satisfied. It is a common method used in functional programming languages like *C,C* + +, etc.

Iterative algorithms use constructs like loops and sometimes other data structures like stacks and queues to solve the problems.

Some problems are suited for recursive and others are suited for iterative. For example, the *Towers of Hanoi* problem can be easily understood in recursive implementation. Every recursive version has an iterative version, and vice versa.

Procedural or Declarative (*n*on-Procedural)

In *declarative* programming languages, we say what we want without having to say how to do it. With *procedural* programming, we have to specify the exact steps to get the result. For example, SQL is more declarative than procedural, because the queries don't specify the steps to produce the result. Examples of procedural languages include: C, PHP, and PERL.

Serial or Parallel or Distributed

In general, while discussing the algorithms we assume that computers execute one instruction at a time. These are called *serial* algorithms.

Parallel algorithms take advantage of computer architectures to process several instructions at a time. They divide the problem into subproblems and serve them to several processors or threads. Iterative algorithms are generally parallelizable.

If the parallel algorithms are distributed on to different machines then we call such algorithms *distributed* algorithms.

Deterministic or Non-Deterministic

Deterministic algorithms solve the problem with a predefined process, whereas *non – deterministic* algorithms guess the best solution at each step through the use of heuristics.

Exact or Approximate

As we have seen, for many problems we are not able to find the optimal solutions. That means, the algorithms for which we are able to find the optimal solutions are called *exact* algorithms. In computer science, if we do not have the optimal solution, we give approximation algorithms.

Approximation algorithms are generally associated with NP-hard problems (refer to the *Complexity Classes* chapter for more details).

16.4 Classification by Design Method

Another way of classifying algorithms is by their design method.

Greedy Method

Greedy algorithms work in stages. In each stage, a decision is made that is good at that point, without bothering about the future consequences. Generally, this means that some *local best* is chosen. It assumes that the local best selection also makes for the *global* optimal solution.

Divide and Conquer

The D & C strategy solves a problem by:

1) Divide: Breaking the problem into sub problems that are themselves smaller instances of the same type of problem.
2) Recursion: Recursively solving these sub problems.
3) Conquer: Appropriately combining their answers.

Examples: merge sort and binary search algorithms.

Dynamic Programming

Dynamic programming (DP) and memoization work together. The difference between DP and divide and conquer is that in the case of the latter there is no dependency among the sub problems, whereas in DP there will be an overlap of sub-problems. By using memoization [maintaining a table for already solved sub problems], DP reduces the exponential complexity to polynomial complexity ($O(n^2)$, $O(n^3)$, etc.) for many problems.

The difference between dynamic programming and recursion is in the memoization of recursive calls. When sub problems are independent and if there is no repetition, memoization does not help, hence dynamic programming is not a solution for all problems.

By using memoization [maintaining a table of sub problems already solved], dynamic

programming reduces the complexity from exponential to polynomial.

Linear Programming

In linear programming, there are inequalities in terms of inputs and *maximizing* (or *minimizing*) some linear function of the inputs. Many problems (example: maximum flow for directed graphs) can be discussed using linear programming.

Reduction [Transform and Conquer]

In this method we solve a difficult problem by transforming it into a known problem for which we have asymptotically optimal algorithms. In this method, the goal is to find a reducing algorithm whose complexity is not dominated by the resulting reduced algorithms. For example, the selection algorithm for finding the median in a list involves first sorting the list and then finding out the middle element in the sorted list. These techniques are also called *transform and conquer.*

16.5 Other Classifications

Classification by Research Area

In computer science each field has its own problems and needs efficient algorithms. Examples: search algorithms, sorting algorithms, merge algorithms, numerical algorithms, graph algorithms, string algorithms, geometric algorithms, combinatorial algorithms, machine learning, cryptography, parallel algorithms, data compression algorithms, parsing techniques, and more.

Classification by Complexity

In this classification, algorithms are classified by the time they take to find a solution based on their input size. Some algorithms take linear time complexity (O(*n*)) and others take exponential time, and some never halt. Note that some problems may have multiple algorithms with different complexities.

Randomized Algorithms

A few algorithms make choices randomly. For some problems, the fastest solutions must involve randomness. Example: Quick Sort.

Branch and Bound Enumeration and Backtracking

These were used in Artificial Intelligence and we do not need to explore these fully. For the Backtracking method refer to the *Recusion and Backtracking* chapter.

Note: In the next few chapters we discuss the Greedy, Divide and Conquer, and Dynamic Programming] design methods. These methods are emphasized because they are used more often than other methods to solve problems.

CHAPTER

17

GREEDY ALGORITHMS

17.1 Introduction

Let us start our discussion with simple theory that will give us an understanding of the Greedy technique. In the game *of Chess*, every time we make a decision about a move, we have to also think about the future consequences. Whereas, in the game of *Tennis* (or *Volleyball*), our action is based on the immediate situation.

This means that in some cases making a decision that looks right at that moment gives the best solution (*Greedy*), but in other cases it doesn't. The Greedy technique is best suited for looking at the immediate situation.

17.2 Greedy Strategy

Greedy algorithms work in stages. In each stage, a decision is made that is good at that point, without bothering about the future. This means that some *local best* is chosen. It assumes that a local good selection makes for a global optimal solution.

17.3 Elements of Greedy Algorithms

The two basic properties of optimal Greedy algorithms are:

1) Greedy choice property
2) Optimal substructure

Greedy choice property

This property says that the globally optimal solution can be obtained by making a locally optimal solution (Greedy). The choice made by a Greedy algorithm may depend on earlier choices but not on the future. It iteratively makes one Greedy choice after another and reduces the given problem to a smaller one.

Optimal substructure

A problem exhibits optimal substructure if an optimal solution to the problem contains optimal solutions to the subproblems. That means we can solve subproblems and build up the solutions to solve larger problems.

17.4 Does Greedy Always Work?

Making locally optimal choices does not always work. Hence, Greedy algorithms will not always give the best solutions. We will see particular examples in the *Problems* section and in the *Dynamic Programming* chapter.

17.5 Advantages and Disadvantages of Greedy Method

The main advantage of the Greedy method is that it is straightforward, easy to understand and easy to code. In Greedy algorithms, once we make a decision, we do not have to spend time re-examining the already computed values. Its main disadvantage is that for many problems there is no greedy algorithm. That means, in many cases there is no guarantee that making locally optimal improvements in a locally optimal solution gives the optimal global solution.

17.6 Greedy Applications

- Sorting: Selection sort, Topological sort
- Priority Queues: Heap sort
- Huffman coding compression algorithm

- Prim's and Kruskal's algorithms
- Shortest path in Weighted Graph [Dijkstra's]
- Coin change problem
- Fractional Knapsack problem
- Disjoint sets-UNION by size and UNION by height (or rank)
- Job scheduling algorithm
- Greedy techniques can be used as an approximation algorithm for complex problems

17.7 Understanding Greedy Technique

For better understanding let us go through an example.

Huffman Coding Algorithm

Definition

Given a set of *n* characters from the alphabet A [each character c ∈ A*]* and their associated frequency *freq*(c), find a binary code for each character c ∈ *A*, such that $\sum_{c \in A}$ freq(c)|binarycode(c)| is minimum, where /binarycode*(c)*/represents the length of binary code of character c. That means the sum of the lengths of all character codes should be minimum [the sum of each character's frequency multiplied by the number of bits in the representation].

The basic idea behind the Huffman coding algorithm is to use fewer bits for more frequently occurring characters. The Huffman coding algorithm compresses the storage of data using variable length codes. We know that each character takes 8 bits for representation. But in general, we do not use all of them. Also, we use some characters more frequently than others. When reading a file, the system generally reads 8 bits at a time to read a single character. But this coding scheme is inefficient. The reason for this is that some characters are more frequently used than other characters. Let's say that the character ′e′ is used 10 times more frequently than the character ′*q*′. It would then be advantageous for us to instead use a 7 bit code for e and a 9 bit code for *q* because that could reduce our overall message length.

On average, using Huffman coding on standard files can reduce them anywhere from 10% to 30% depending on the character frequencies. The idea behind the character coding is to give longer binary codes for less frequent characters and groups of characters. Also, the character coding is constructed in such a way that no two character codes are prefixes of each other.

An Example

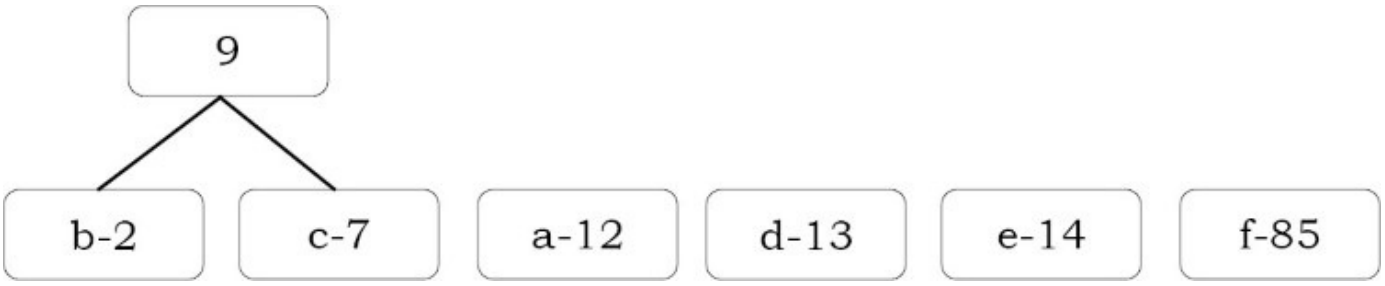
Let's assume that after scanning a file we find the following character frequencies:

Character	Frequency
a	12
b	2
c	7
d	13
e	14
f	85

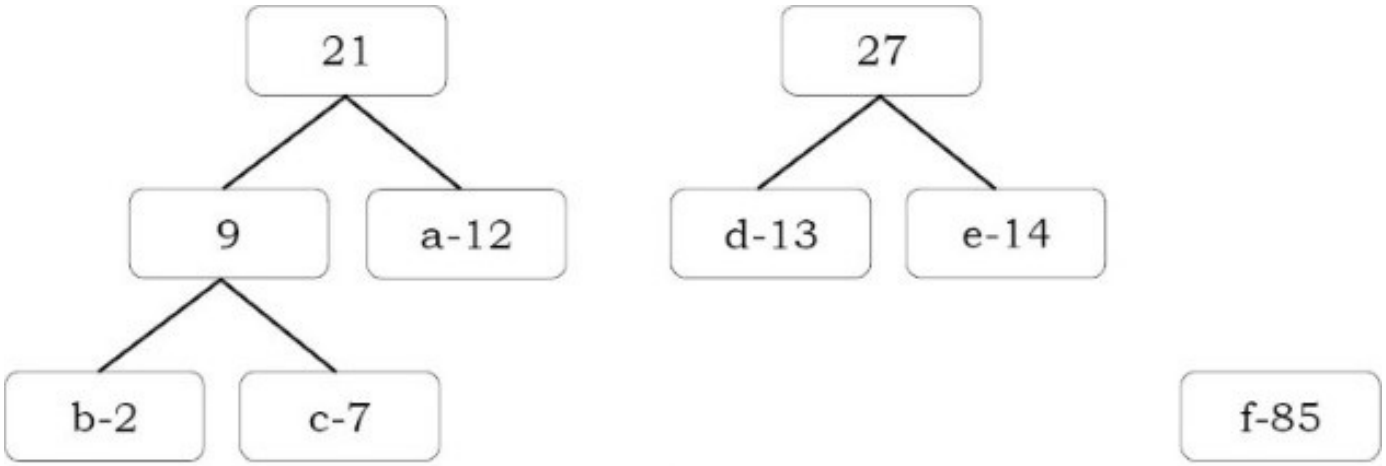
Given this, create a binary tree for each character that also stores the frequency with which it occurs (as shown below).

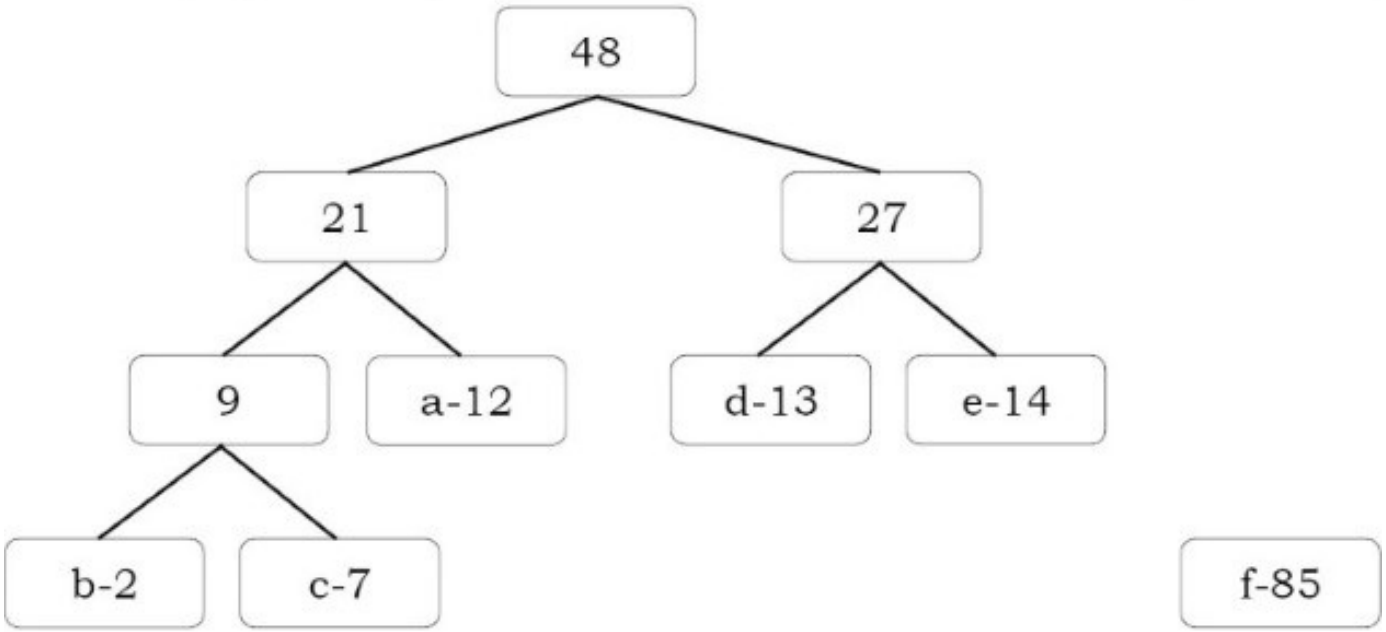
The algorithm works as follows: In the list, find the two binary trees that store minimum frequencies at their nodes.

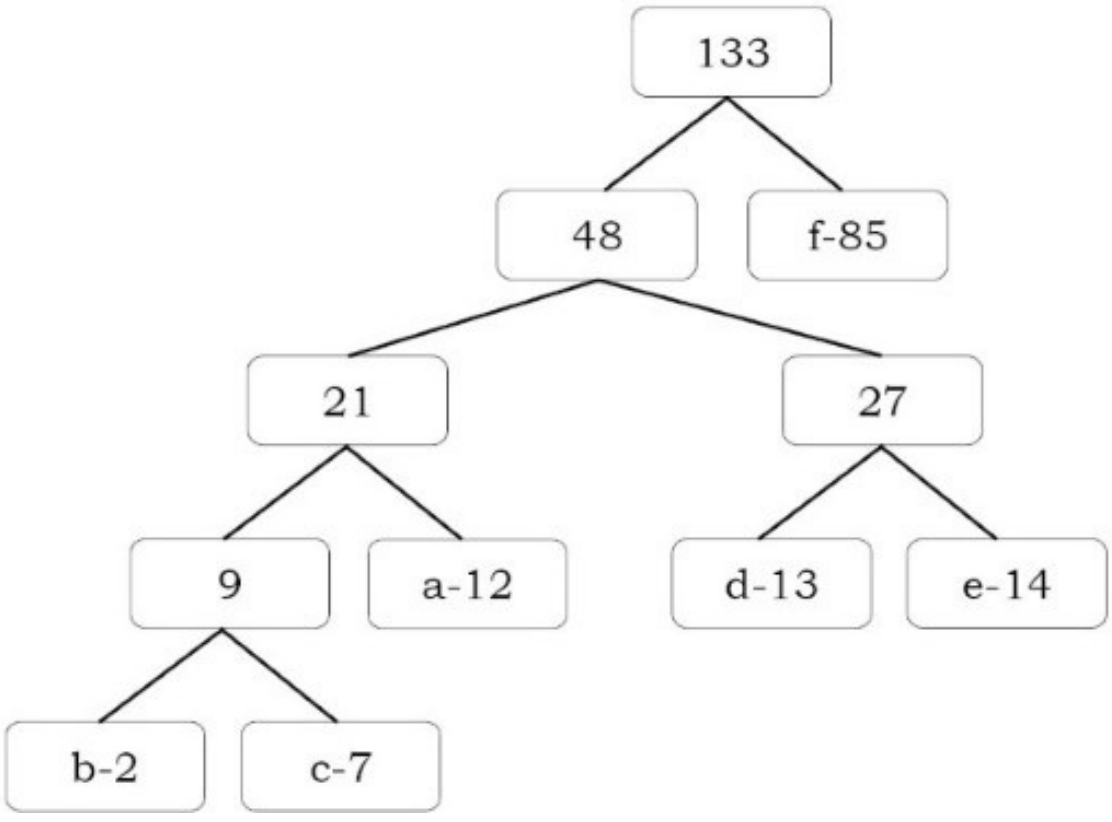
Connect these two nodes at a newly created common node that will store no character but will store the sum of the frequencies of all the nodes connected below it. So our picture looks like this:



Repeat this process until only one tree is left:


21
27
9
a-12
d-13
e-14
b-2
c-7
f-85


48
21
27
9
a-12
d-13
e-14
b-2
c-7
f-85


133
48
f-85
21
27
9
a-12
d-13
e-14
b-2
c-7

Once the tree is built, each leaf node corresponds to a letter with a code. To determine the code for a particular node, traverse from the root to the leaf node. For each move to the left, append a 0 to the code, and for each move to the right, append a 1. As a result, for the above generated tree, we get the following codes:

Letter	Code
a	001
b	0000
c	0001
d	010
e	011
f	1

Calculating Bits Saved

Now, let us see how many bits that Huffman coding algorithm is saving. All we need to do for this calculation is see how many bits are originally used to store the data and subtract from that the number of bits that are used to store the data using the Huffman code. In the above example, since we have six characters, let's assume each character is stored with a three bit code. Since there are 133 such characters (multiply total frequencies by 3), the total number of bits used is 3 * 133 = 399. Using the Huffman coding frequencies we can calculate the new total number of bits used:

Letter	Code	Frequency	Total Bits
a	001	12	36
b	0000	2	8
c	0001	7	28
d	010	13	39
e	011	14	42
f	1	85	85
Total			238

Thus, we saved 399 – 238 = 161 bits, or nearly 40% of the storage space.

```
HuffmanCodingAlgorithm(int A[], int n) {
    Initialize a priority queue, PQ, to contain the n elements in A;
    struct BinaryTreeNode *temp;
    for (i = 1; i<n; i++) {
        temp  = (struct *)malloc(sizeof(BinaryTreeNode));
        temp→left = Delete-Min(PQ);
        temp→right = Delete-Min(PQ);
        temp→data = temp→left→data + temp→right→data;
        Insert temp to PQ;
    }
    return PQ;
}
```

Time Complexity: O(*nlogn*), since there will be *one* build_heap, 2*n* – 2 delete_mins, and *n* – 2 inserts, on a priority queue that never has more than n elements. Refer to the *Priority Queues* chapter for details.

17.8 Greedy Algorithms: Problems & Solutions

Problem-1 Given an array *F* with size n. Assume the array content *F*[*i*] indicates the length of the i^{th} file and we want to merge all these files into one single file. Check whether the following algorithm gives the best solution for this problem or not?

Algorithm: Merge the files contiguously. That means select the first two files and merge them. Then select the output of the previous merge and merge with the third file, and keep going...

Note: Given two files *A* and *B* with sizes *m* and n, the complexity of merging is O(*m* + *n*).

Solution: This algorithm will not produce the optimal solution. For a counter example, let us consider the following file sizes array.

$$F = \{10,5,100,50,20,15\}$$

As per the above algorithm, we need to merge the first two files (10 and 5 size files), and as a result we get the following list of files. In the list below, 15 indicates the cost of merging two files with sizes 10 and 5.

$$\{15,100,50,20,15\}$$

Similarly, merging 15 with the next file 100 produces: {115,50,20,15}. For the subsequent steps

the list becomes

{165,20,15}, {185,15}

Finally,

{200}

The total cost of merging = Cost of all merging operations = 15 + 115 + 165 + 185 + 200 = 680.

To see whether the above result is optimal or not, consider the order: {5,10,15,20,50,100}. For this example, following the same approach, the total cost of merging = 15 + 30 + 50 + 100 + 200 = 395. So, the given algorithm is not giving the best (optimal) solution.

Problem-2 Similar to Problem-1, does the following algorithm give the optimal solution?

> **Algorithm:** Merge the files in pairs. That means after the first step, the algorithm produces the $n/2$ intermediate files. For the next step, we need to consider these intermediate files and merge them in pairs and keep going.
>
> **Note:** Sometimes this algorithm is called 2-way merging. Instead of two files at a time, if we merge K files at a time then we call it K-way merging.

Solution: This algorithm will not produce the optimal solution and consider the previous example for a counter example. As per the above algorithm, we need to merge the first pair of files (10 and 5 size files), the second pair of files (100 and 50) and the third pair of files (20 and 15). As a result we get the following list of files.

{15,150,35}

Similarly, merge the output in pairs and this step produces [below, the third element does not have a pair element, so keep it the same]:

{165,35}

Finally,

{185}

The total cost of merging = Cost of all merging operations = 15 + 150 + 35 + 165 + 185 = 550. This is much more than 395 (of the previous problem). So, the given algorithm is not giving the best (optimal) solution.

Problem-3 In Problem-1, what is the best way to merge *all the files* into a single file?

Solution: Using the Greedy algorithm we can reduce the total time for merging the given files. Let us consider the following algorithm.

Algorithm:

1. Store file sizes in a priority queue. The key of elements are file lengths.
2. Repeat the following until there is only one file:
    a. Extract two smallest elements *X* and *Y*.
    b. Merge *X* and *Y* and insert this new file in the priority queue.

Variant of same algorithm:

1. Sort the file sizes in ascending order.
2. Repeat the following until there is only one file:
    a. Take the first two elements (smallest) *X* and *Y*.
    b. Merge *X* and *Y* and insert this new file in the sorted list.

To check the above algorithm, let us trace it with the previous example. The given array is:

$$F = \{10,5,100,50,20,15\}$$

As per the above algorithm, after sorting the list it becomes: {5,10,15,20,50,100}. We need to merge the two smallest files (5 and 10 size files) and as a result we get the following list of files. In the list below, 15 indicates the cost of merging two files with sizes 10 and 5.

$$\{15,15,20,50,100\}$$

Similarly, merging the two smallest elements (15 and 15) produces: {20,30,50,100}. For the subsequent steps the list becomes

$$\{50,50,100\} \text{ // merging 20 and 30}$$
$$\{100,100\} \text{ // merging 20 and 30}$$

Finally,

$$\{200\}$$

The total cost of merging = Cost of all merging operations = 15 + 30 + 50 + 100 + 200 = 395. So, this algorithm is producing the optimal solution for this merging problem.

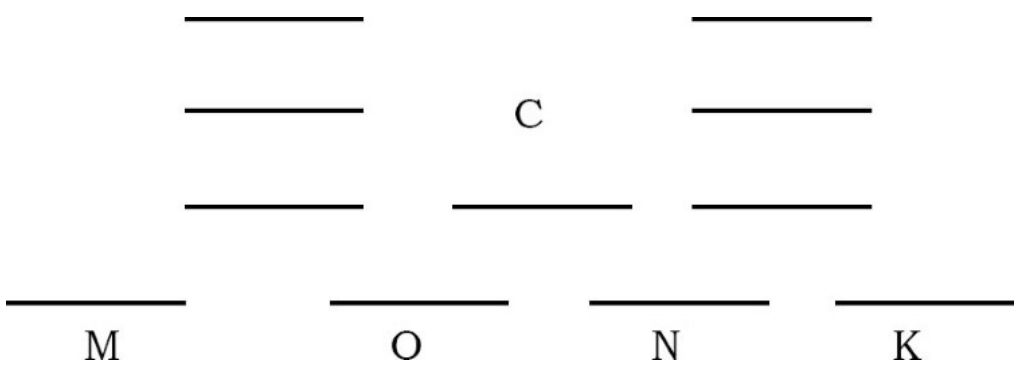
Time Complexity: O(*nlogn*) time using heaps to find best merging pattern plus the optimal cost of merging the files.

Problem-4 **Interval Scheduling Algorithm:** Given a set of *n* intervals $S = \{(start_i, end_j) | 1 \le i \le n\}$. Let us assume that we want to find a maximum subset *S′* of *S* such that no pair of intervals in *S′* overlaps. Check whether the following algorithm works or not.

Algorithm:

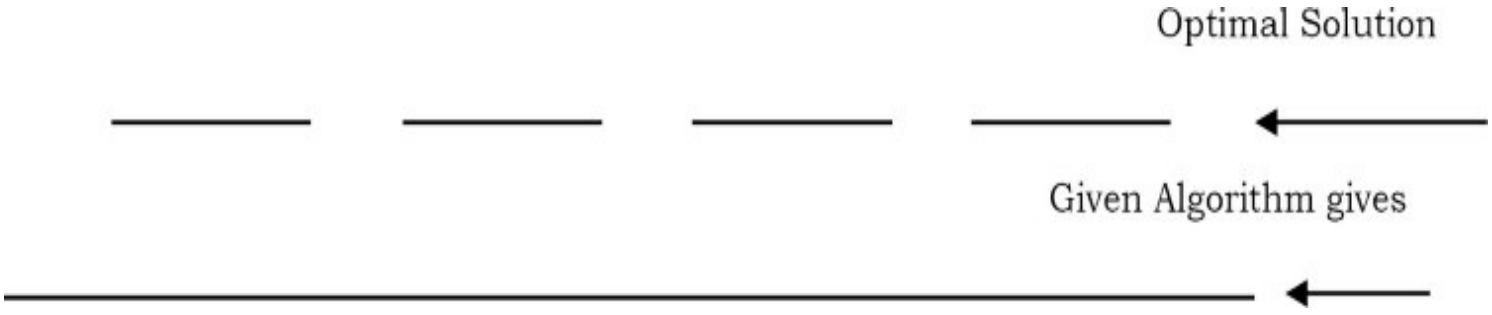
```
while (S is not empty) {
        Select the interval I that overlaps the least number of other intervals.
        Add I to final solution set S'.
        Remove all intervals from S that overlap with I.
}
```

Solution: This algorithm does not solve the problem of finding a maximum subset of non-overlapping intervals. Consider the following intervals. The optimal solution is {*M*,*O*,*N*,*K*}. However, the interval that overlaps with the fewest others is *C*, and the given algorithm will select *C* first.



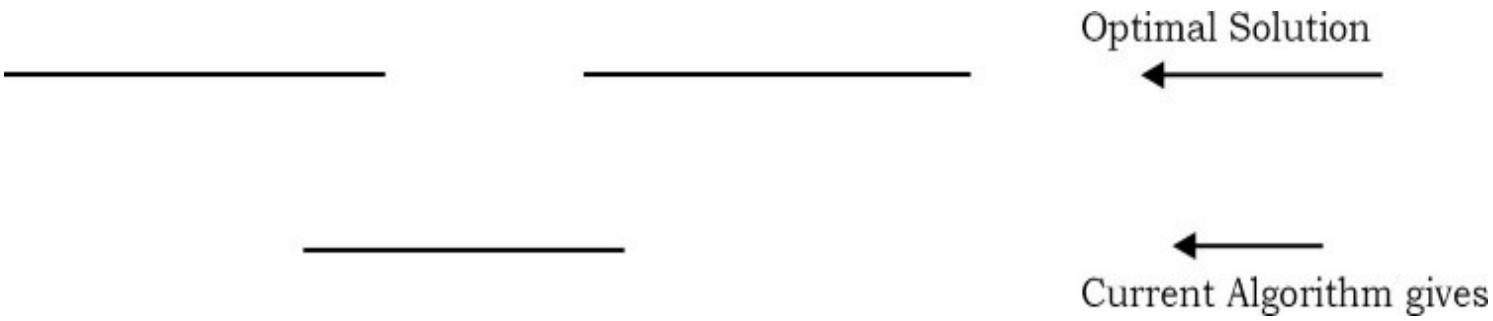
Problem-5 In Problem-4, if we select the interval that starts earliest (also not overlapping with already chosen intervals), does it give the optimal solution?

Solution: No. It will not give the optimal solution. Let us consider the example below. It can be seen that the optimal solution is 4 whereas the given algorithm gives 1.



Problem-6 In Problem-4, if we select the shortest interval (but it is not overlapping the already chosen intervals), does it give the optimal solution?

Solution: This also will not give the optimal solution. Let us consider the example below. It can be seen that the optimal solution is 2 whereas the algorithm gives 1.



Problem-7 For Problem-4, what is the optimal solution?

Solution: Now, let us concentrate on the optimal greedy solution.

Algorithm:

Sort intervals according to the right-most ends [end times];
for every consecutive interval {
- If the left-most end is after the right-most end of the last selected interval then we select this interval
- Otherwise we skip it and go to the next interval

}

Time complexity = Time for sorting + Time for scanning = O(*nlogn* + *n*) = O(*nlogn*).

Problem-8 Consider the following problem.

Input: $S = \{(start_i,end_i) | 1 \le i \le n\}$ of intervals. The interval $(start_i,end_i)$ we can treat as a request for a room for a class with time start; to time end_i.

Output: Find an assignment of classes to rooms that uses the fewest number of rooms.

Consider the following iterative algorithm. Assign as many classes as possible to the first room, then assign as many classes as possible to the second room, then assign as many classes as possible to the third room, etc. Does this algorithm give the best solution?

Note: In fact, this problem is similar to the interval scheduling algorithm. The only difference is the application.

Solution: This algorithm does not solve the interval-coloring problem. Consider the following intervals: